18. Hardworking Subroutines

Machine language programs take considerable effort to code and get working because they involve large numbers of instructions. On the other hand, they are the most efficient form of program. A good way to increase the power of a computer is to build a collection of machine language subroutines that can be combined in different ways to form executing programs. Some subroutines do commonly useful tasks which can be used in many programs. In writing such subroutines, it is important to keep them as general as possible. Avoid combining several processing tasks together into one subroutine, restricting it to fewer situations.

There are many ways to provide input data and locations for output data for subroutines. Input data can be left in processor registers. One example would be the Monitor subroutine at $FE93, described in Figure 14.2. The input and output result are transmitted in the accumulator. Another useful example is the 'time delay' subroutine of Figure 18.1, with input data left in register X. It passes time by executing an inner loop 256 times and repeating this the number of times specified by X. Place calls to this subroutine in a program to slow down the action, so you can actually see something happening. If X is in use at the point of the desired delay, then you must save its contents somewhere before calling 'time delay' and restore its contents afterwards. The location $ØØFD, which is wiped out by the call to 'time delay', does not have to be initially set to any particular value before the call unless an exact time is required.

```
          ;TIME DELAY OF ABOUT 2265.X CYCLES
          ;CLEARS $FD
EA        TIME NOP        ; EACH NOP ADDS 512 CYCLES TO THE LOOP
C6 FD          DEC $FD    ; 256 TIMES, AFTER THE FIRST
DØ FB          BNE TIME
88             DEX        ; COUNTS EXECUTED LOOPS
DØ F8          BNE TIME
6Ø             RTS
```

Time Delay Subroutine

Figure 18.1

```
      ;SHIFT DISPLAY FIELD LEFT, STORE A INTO RIGHTMOST CHARACTER
      ;FIELD ADDRESS AT $F9,$F8,LENGTH-1 AT $FA
      ;AFFECTS Y
AØ ØØ SCROLL  LDY #Ø          ; STRING POINTER
48            PHA             ; SAVE INSERT
C8    SHIFT   INY             ; I+1
B1 F8         LDA ($F8),Y     ; MOVE (I+1)ST POSITION
88            DEY             ; I
91 F8         STA ($F8),Y     ; TO (I)TH POSITION
C8            INY             ; ADVANCE I
C4 FA         CPY $FA         ; LOOP TEST VALUE
DØ F5         BNE SHIFT
68            PLA             ; RESTORE AND
91 F8         STA ($F8),Y     ; INSERT INTO RIGHTMOST
6Ø            RTS
```

Character Scroll

Figure 18.2

Another way of passing data to subroutines is illustrated in Figure 18.2. The address and length of the 'scroll' field are placed in zero page locations. The LDA and STA instructions use an addressing mode called indirect indexed, in which the instruction identifies the zero page location of the address of the scroll field, and the contents of Y are added to form the effective address of a byte in the field. The scroll field address is a two-byte address which can be changed to relocate the scroll field anywhere in the display. The scroll subroutine does a big part of good listener's task. See if you can rewrite "good listener" to use the scroll subroutine. Don't forget to load appropriate values into locations ØØF8, ØØF9 and ØØFA.

You can experiment with the time delay subroutine by loading it and inserting the instructions

```
LDX  #_____
JSR  TIME
```

within the scroll subroutine's loop. Remember to subtract five from the relative addressing displacement in

```
BNE  SHIFT
```

to adjust for the insertion. Adjust the immediate value loaded into X for a pleasing 'ripple' effect as you enter data.

19. Two-Way Arithmetic

Many computers can do decimal arithmetic directly, in addition to binary arithmetic. As you know, converting numbers between the binary and decimal number systems is not so easy. A program that receives decimal data and outputs decimal results, while doing all internal computations in binary, can have a lot of converting to do. Decimal arithmetic, though not as efficient as binary arithmetic, can be better in many situations as an alternative to conversions in both directions.

The 65Ø2 processor has a switch that can be thrown to make add and substract operations produce decimal, rather than binary results. In this mode of operation, byte operands which are valid BCD (Binary Coded Decimal) inputs are combined into valid BCD sums and differences, with the carry flag representing carries or borrows of ten. See Section 5 to review BCD. Although a ten's complement corresponding to the two's complement does exist for the representation of negative numbers, decimal arithmetic is usually done with positive numbers.

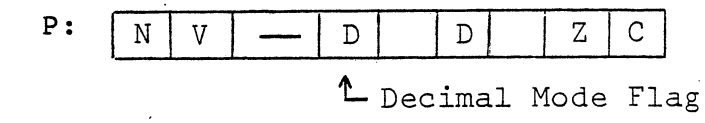
A decimal mode flag is the means of controlling the arithmetic mode of the processor and indicating which mode the processor is in. One-byte instructions set this flag to enter decimal mode and clear it to return to binary arithmetic. These instructions and the position of the decimal mode flag (D) in the P register which is pushed onto the stack by the PHP instruction, are shown in Figure 19.1.

Decimal mode arithmetic can be explored by changing the first

instruction in "total mystery" (original or your extended version) to an SED instruction. This makes the program execute in decimal mode. The Monitor clears the decimal mode flag as it begins execution, but care must be taken when entering the Monitor at other points that the decimal mode flag is cleared.

You might enjoy the program of Figure 19.2, which requires several of the subroutines you have seen in previous sections. The program converts hexadecimal numbers to decimal by using decimal arithmetic. Minor changes allow it to convert decimal numbers to hexadecimal by doing the same operations in binary arithmetic. A necessary multiplication is carried out by adding repeatedly; not the most efficient method, but simple enough to make the two-way arithmetic idea work. More efficient multiplication methods involving adding and shifting are spelled out in many references. The bibliography at the end of this manual will lead you to specific methods and coded subroutines.

Mnemonic	Explanation	Opcode
CLD	Clear Decimal Mode Flag	D8
SED	Set Decimal Mode Flag	F8



The Decimal Mode Flag

Figure 19.1

```
;DECIMAL-HEX CONVERTER:
;FOR DECIMAL TO HEX, START WITH CLD AND CNT+1=9
;FOR HEX TO DECIMAL, START WITH SED AND CNT+1=15
;
1000 D8        START   CLD           ; CLD OR SED
1001 A9 20     REPEAT  LDA #$20      ; ASCII BLANK
1003 A0 00             LDX #0        ; ZERO
1005 9D C6 D0  CLEAR   STA $D0C6,X   ; CLEAR ENTRY FIELD
1008 94 E0             STY $E0,X     ; CLEAR ACCUMULATOR
100A CA                DEX
100B 10 F8             BPL CLEAR
100D A2 E0     AOUT    LDX #$E0      ; INPUT ADDRESS
100F A0 40             LDY #$40      ; DISPLAY POSTION
1010 20 -- --          JSR HEXTV     ; ACCUMULATOR DISPLAY, FIG 16.3
1013 20 ED FE          JSR $FEED     ; GET DIGIT
1017 48                PHA           ; SAVE ASCII
1018 20 93 FE          JSR $FE93     ; STRIP TO DIGIT, FIG 14.2
101B 10 03             BPL DIG       ; BRANCH IF DIGIT
101D 68                PLA           ; IF NOT, DISCARD IT
101E 50 E1             BVC REPEAT    ;
1020 48        DIGIT   PHA           ; SAVE STRIPPED FOR ADD
1021 A2 07             LDX #7
1023 B5 E0     COPY    LDA $E0,X     ; COPY FOR MULTIPLY
1025 95 E8             STA $E8,X
1027 A2 09     CNT     LDX #9
1029 A0 07     MULT    LDY #7        ; MULTIPLY BY ADDING
102B 20 -- --          JSR ADDN      ; FIGURE 15.4
102E CA                DEX
102F D0 F8             BNE MULT
1031 68                PLA           ; PULL STRIPPED DIGIT
1032 A2 07             LDX #7
1034 20 -- --          JSR ADD1      ; ADD TO ACCUMULATOR
1037 68                PLA           ; PULL DIGIT ASCII
1038 20 -- --          JSR SCROLL    ; ROLL INTO INPUT DISPLAY, FIG 18.2
103B 4C 0D 10          JMP AOUT      ; DISPLAY NEW ACCUMULATOR
```

Decimal-Hexadecimal Converter

Figure 19.2

The machine code addresses in Figure 19.2 assume the program is loaded into memory beginning at location $1000. Actually it can be loaded anywhere in RAM, provided the absolute address in the last instruction is adjusted to the location of the instruction labeled AOUT. This one dependence on the load address of the program can be removed by replacing the last instruction with a conditional branch that is known to branch from the known value of its flag. This strategy was followed in the instruction shown at $101E.

Blanks occur in some absolute address positions in the program, corresponding to subroutine calls. The subroutines are those defined in this manual. You can load them anywhere in RAM and fill in their absolute addresses (with reversed bytes) to make a complete program.

Adjusting a program to its loading location is called relocation. Filling in addresses to reflect the location of other routines and data is called linking or resolving registers. Many computer systems have a system program, called a loader, which loads, relocates and links the user's programs.

2Ø. A Sparkling Finish

You have almost reached the end of this guide, but that is no reason to slow down in enabling your computer to do new useful and amusing things, through machine language programming.

We have explored almost every 65Ø2 instruction, but there is an addressing mode we have not encountered. It is called indexed indirect. In this mode, the index register X selects an address from a block of reversed-byte addresses in the zero page. The starting byte of the block is designated by the second byte of the indexed instruction. In the symbolic assembler language form the indexed indirect mode is indicated by an operand of the form

(zero-page-address,X)

The indexed indirect addressing mode is useful for accessing bytes, not in increasing or decreasing order. The program of Figure 2Ø.1 illustrates the situation. Select a set of display cells scattered 'randomly' around on the screen. List them in a block of reversed-byte addresses, not in any particular order. Load the program somewhere beyond the block of addresses and load the 'time delay' subroutine along with it. Link the 'time delay' into your program. Place the number of addresses in location $ØØØ1 and adjust the timing constant for a pleasing effect.

In the sparkle program, the 'stars' stay off most of the time. To make a 'twinkle, twinkle, little star' version, reverse the star and blank characters $E8 and $2Ø.

```
            ;SPARKLING FINISH
            ;PLACE ADDRESSES OF SPARKLE DISPLAY POINTS IN LOCATIONS
                  Ø2,Ø3 THROUGH 2N,2N+1 IN REVERSED BYTE FORM
            ;LOAD A TIMING CONSTANT FOR THE 'ON' TIME IN $ØØ
            ;LOAD N INTO $Ø1
            ;
A6 Ø1       BEGIN LDX 1         ;
A9 E8       LOOP  LDA #$E8      ; A 'STAR'
81 Ø2             STA ($Ø2,X)   ; THE STAR APPEARS
8A                TXA           ; SAVE FOR TIME
A6 ØØ             LDX Ø
2Ø -- --          JSR TIME      ; FLASH A STAR
AA                TAX           ; RESTORE STAR POINTER
A9 2Ø             LDA #$2Ø      ; BLANK
81 Ø2             STA ($Ø2,X)   ; TURN STAR OFF
CA                DEX           ;
CA                DEX           ; INDEX NEXT STAR LOCATION
DØ ED             BNE LOOP      ; FLASH NEXT STAR
FØ E9             BEQ BEGIN     ; CYCLE THROUGH STARS AGAIN
```

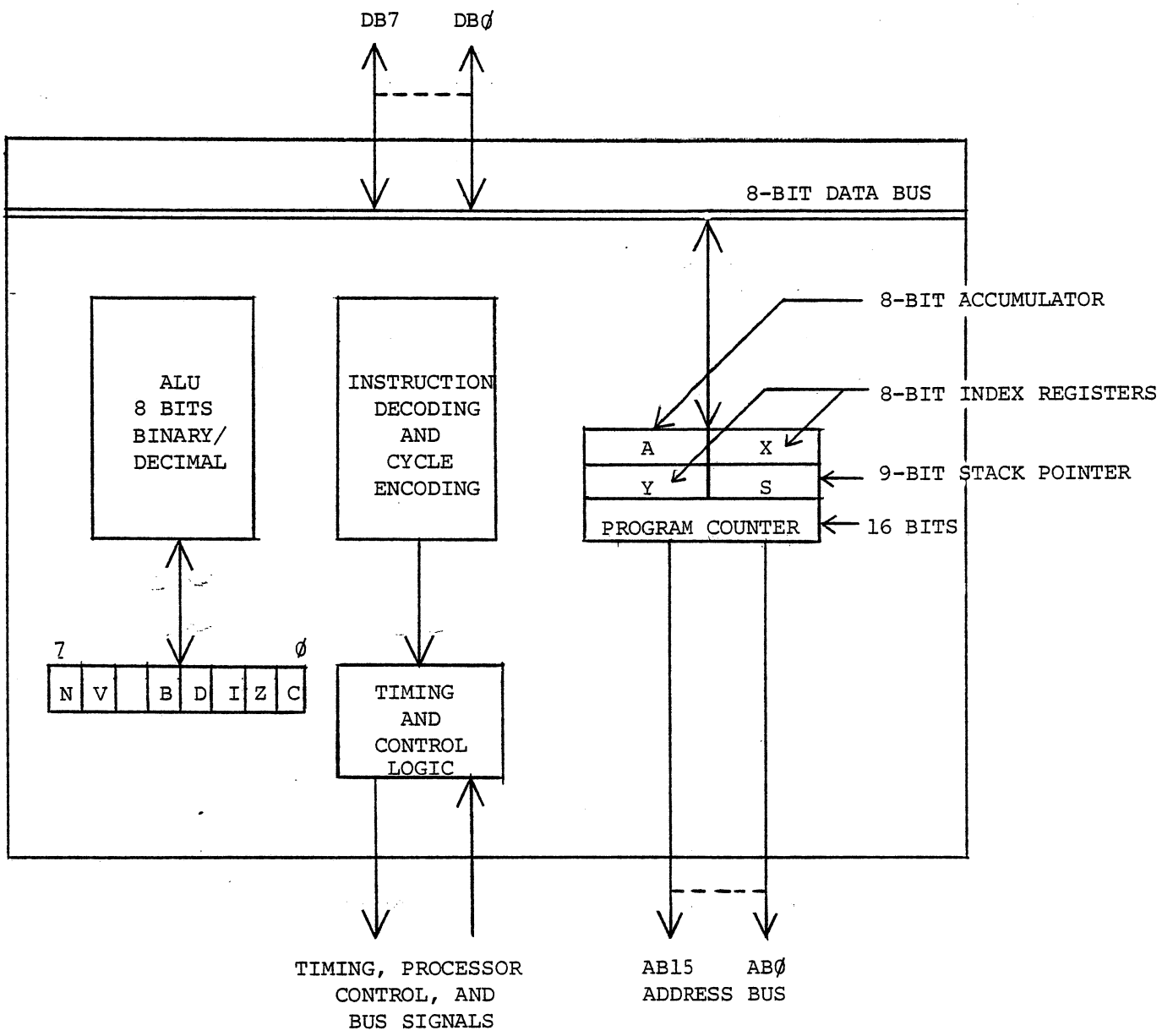
A Sparkling Program

Figure 2Ø.1

APPENDICES

ASCII CHARACTER CODES

Code	Char	Code	Char	Code	Char
00	NUL	2B	+	56	V
01	SOH	2C	,	57	W
02	STX	2D	-	58	X
03	ETX	2E	.	59	Y
04	EOT	2F	/	5A	Z
05	ENQ	30	0	5B	[
06	ACK	31	1	5C	\
07	BEL	32	2	5D	]
08	BS	33	3	5E	^
09	HT	34	4	5F	_
0A	LF	35	5	60	`
0B	VT	36	6	61	a
0C	FF	37	7	62	b
0D	CR	38	8	63	c
0E	SO	39	9	64	d
0F	SI	3A	:	65	e
10	DLE	3B	;	66	f
11	DC1	3C	<	67	g
12	DC2	3D	=	68	h
13	DC3	3E	>	69	i
14	DC4	3F	?	6A	j
15	NAK	40	@	6B	k
16	SYN	41	A	6C	l
17	ETB	42	B	6D	m
18	CAN	43	C	6E	n
19	EM	44	D	6F	o
1A	SUB	45	E	70	p
1B	ESC	46	F	71	q
1C	FS	47	G	72	r
1D	GS	48	H	73	s
1E	RS	49	I	74	t
1F	US	4A	J	75	u
20	SP	4B	K	76	v
21	!	4C	L	77	w
22	"	4D	M	78	x
23	#	4E	N	79	y
24	$	4F	O	7A	z
25	%	50	P	7B	{
26	&	51	Q	7C	|
27	'	52	R	7D	}
28	(	53	S	7E	~
29	)	54	T	7F	DEL
2A	*	55	U		



65Ø2 MICROPROCESSOR ARCHITECTURE

6502 Instruction Set - Mnemonic List

ADC	Add Memory to Accumulator with Carry
AND	"AND" Memory with Accumulator
ASL	Shift Left One Bit (Memory or Accumulator)
BCC	Branch on Carry Clear
BCS	Branch on Carry Set
BEQ	Branch on Result Zero
BIT	Test Bits in Memory with Accumulator
BMI	Branch on Result Minus
BNE	Branch on Result not Zero
BPL	Branch on Result Plus
BRK	Force Break
BVC	Branch on Overflow Clear
BVS	Branch on Overflow Set
CLC	Clear Carry Flag
CLD	Clear Decimal Mode
CLI	Clear Interrupt Disable Bit
CLV	Clear Overflow Flag
CMP	Compare Memory and Accumulator
CPX	Compare Memory and Index X
CPY	Compare Memory and Index Y
DEC	Decrement Memory by One
DEX	Decrement Index X by One
DEY	Decrement Index Y by One
EOR	"Exclusive Or" Memory with Accumulator
INC	Increment Memory by One
INX	Increment Index X by One
INY	Increment Index Y by One
JMP	Jump to New Location
JSR	Jump to New Location Saving Return Address
LDA	Load Accumulator with Memory
LDX	Load Index X with Memory
LDY	Load Index Y with Memory
LSR	Shift Right One Bit (Memory or Accumulator)
NOP	No Operation
ORA	"OR" Memory with Accumulator
PHA	Push Accumulator on Stack
PHP	Push Processor Status on Stack
PLA	Pull Accumulator from Stack
PLP	Pull Processor Status from Stack

6502 Instruction Set - Mnemonic List (continued)

ROL	Rotate One Bit Left (Memory or Accumulator)
ROR	Rotate One Bit Right (Memory or Accumulator)
RTI	Return from Interrupt
RTS	Return from Subroutine
SBC	Subtract Memory from Accumulator with Borrow
SEC	Set Carry Flag
SED	Set Decimal Mode
SEI	Set Interrupt Disable Status
STA	Store Accumulator in Memory
STX	Store Index X in Memory
STY	Store Index Y in Memory
TAX	Transfer Accumulator to Index X
TAY	Transfer Accumulator to Index Y
TSX	Transfer Stack Pointer to Index X
TXA	Transfer Index X to Accumulator
TXS	Transfer Index X to Stack Pointer
TYA	Transfer Index Y to Accumulator

65Ø2 Instruction Set - Hex Listing

ØØ - BRK
Ø1 - ORA - (Indirect,X)
Ø2 - Future Expansion
Ø3 - Future Expansion
Ø4 - Future Expansion
Ø5 - ORA - Zero Page
Ø6 - ASL - Zero Page
Ø7 - Future Expansion
Ø8 - PHP
Ø9 - ORA - Immediate
ØA - ASL - Accumulator
ØB - Future Expansion
ØC - Future Expansion
ØD - ORA - Absolute
ØE - ASL - Absolute
ØF - Future Expansion
1Ø - BPL
11 - ORA - (Indirect),Y
12 - Future Expansion
13 - Future Expansion
14 - Future Expansion
15 - ORA - Zero Page,X
16 - ASL - Zero Page,X
17 - Future Expansion
18 - CLC
19 - ORA - Absolute,Y
1A - Future Expansion
1B - Future Expansion
1C - Future Expansion
1D - ORA - Absolute, X
1E - ASL - Absolute, X
1F - Future Expansion

2Ø - JSR
21 - AND - (Indirect,X)
22 - Future Expansion
23 - Future Expansion
24 - BIT - Zero Page
25 - AND - Zero Page
26 - ROL - Zero Page
27 - Future Expansion
28 - PLP
29 - AND - Immediate
2A - ROL - Accumulator
2B - Future Expansion
2C - BIT - Absolute
2D - AND - Absolute
2E - ROL - Absolute
2F - Future Expansion
3Ø - BMI
31 - AND - (Indirect),Y
32 - Future Expansion
33 - Future Expansion
34 - Future Expansion
35 - AND - Zero Page,X
36 - ROL - Zero Page,X
37 - Future Expansion
38 - SEC
39 - AND - Absolute,Y
3A - Future Expansion
3B - Future Expansion
3C - Future Expansion
3D - AND - Absolute,X
3E - ROL - Absolute,X
3F - Future Expansion

40 - RTI
41 - EOR - (Indirect,X)
42 - Future Expansion
43 - Future Expansion
44 - Future Expansion
45 - EOR - Zero Page
46 - LSR - Zero Page
47 - Future Expansion
48 - PHA
49 - EOR - Immediate
4A - LSR - Accumulator
4B - Future Expansion
4C - JMP - Absolute
4D - EOR - Absolute
4E - LSR - Absolute
4F - Future Expansion
50 - BVC
51 - EOR - (Indirect),Y
52 - Future Expansion
53 - Future Expansion
54 - Future Expansion
55 - EOR - Zero Page,X
56 - LSR - Zero Page,X
57 - Future Expansion
58 - CLI
59 - EOR - Absolute,Y
5A - Future Expansion
5B - Future Expansion
5C - Future Expansion
5D - EOR - Absolute,X
5E - LSR - Absolute,X
5F - Future Expansion

60 - RTS
61 - ADC - (Indirect,X)
62 - Future Expansion
63 - Future Expansion
64 - Future Expansion
65 - ADC - Zero Page
66 - ROR - Zero Page
67 - Future Expansion
68 - PLA
69 - ADC - Immediate
6A - ROR - Accumulator
6B - Future Expansion
6C - JMP - Indirect
6D - ADC - Absolute
6E - ROR - Absolute
6F - Future Expansion
70 - BVS
71 - ADC - (Indirect),Y
72 - Future Expansion
73 - Future Expansion
74 - Future Expansion
75 - ADC - Zero Page,X
76 - ROR - Zero Page,X
77 - Future Expansion
78 - SEI
79 - ADC - Absolute,Y
7A - Future Expansion
7B - Future Expansion
7C - Future Expansion
7D - ADC - Absolute,X
7E - ROR - Absolute,X
7F - Future Expansion

65Ø2 Instruction Set - Hex Listing (continued)

8Ø - Future Expansion
81 - STA - (Indirect,X)
82 - Future Expansion
83 - Future Expansion
84 - STY - Zero Page
85 - STA - Zero Page
86 - STX - Zero Page
87 - Future Expansion
88 - DEY
89 - Future Expansion
8A - TXA
8B - Future Expansion
8C - STY - Absolute
8D - STA - Absolute
8E - STX - Absolute
8F - Future Expansion
9Ø - BCC
91 - STA - (Indirect),Y
92 - Future Expansion
93 - Future Expansion
94 - STY - Zero Page,X
95 - STA - Zero Page,X
96 - STX - Zero Page,Y
97 - Future Expansion
98 - TYA
99 - STA - Absolute,Y
9A - TXS
9B - Future Expansion
9C - Future Expansion
9D - STA - Absolute,X
9E - Future Expansion
9F - Future Expansion

AØ - LDY - Immediate
A1 - LDA - (Indirect,X)
A2 - LDX - Immediate
A3 - Future Expansion
A4 - LDY - Zero Page
A5 - LDA - Zero Page
A6 - LDX - Zero Page
A7 - Future Expansion
A8 - TAY
A9 - LDA - Immediate
AA - TAX
AB - Future Expansion
AC - LDY - Absolute
AD - LDA - Absolute
AE - LDX - Absolute
AF - Future Expansion
BØ - BCS
B1 - LDA - (Indirect),Y
B2 - Future Expansion
B3 - Future Expansion
B4 - LDY - Zero Page,X
B5 - LDA - Zero Page,X
B6 - LDX - Zero Page,Y
B7 - Future Expansion
B8 - CLV
B9 - LDA - Absolute,Y
BA - TSX
BB - Future Expansion
BC - LDY - Absolute,X
BD - LDA - Absolute,X
BE - LDX - Absolute,Y
BF - Future Expansion

C0 - CPY - Immediate
C1 - CMP - (Indirect,X)
C2 - Future Expansion
C3 - Future Expansion
C4 - CPY - Zero Page
C5 - CMP - Zero Page
C6 - DEC - Zero Page
C7 - Future Expansion
C8 - INY
C9 - CMP - Immediate
CA - DEX
CB - Future Expansion
CC - CPY - Absolute
CD - CMP - Absolute
CE - DEC - Absolute
CF - Future Expansion
D0 - BNE
D1 - CMP - (Indirect),Y
D2 - Future Expansion
D3 - Future Expansion
D4 - Future Expansion
D5 - CMP - Zero Page,X
D6 - DEC - Zero Page,X
D7 - Future Expansion
D8 - CLD
D9 - CMP - Absolute,Y
DA - Future Expansion
DB - Future Expansion
DC - Future Expansion
DD - CMP - Absolute,X
DE - DEC - Absolute,X
DF - Future Expansion

E0 - CPX - Immediate
E1 - SBC - (Indirect,X)
E2 - Future Expansion
E3 - Future Expansion
E4 - CPX - Zero Page
E5 - SBC - Zero Page
E6 - INC - Zero Page
E7 - Future Expansion
E8 - INX
E9 - SBC - Immediate
EA - NOP
EB - Future Expansion
EC - CPX - Absolute
ED - SBC - Absolute
EE - INC - Absolute
EF - Future Expansion
F0 - BEQ
F1 - SBC - (Indirect),Y
F2 - Future Expansion
F3 - Future Expansion
F4 - Future Expansion
F5 - SBC - Zero Page,X
F6 - INC - Zero Page,X
F7 - Future Expansion
F8 - SED
F9 - SBC - Absolute,Y
FA - Future Expansion
FB - Future Expansion
FC - Future Expansion
FD - SBC - Absolute,X
FE - INC - Absolute,X
FF - Future Expansion

OPCODE TABLE

MSD \ LSD	Ø	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Ø	BRK	ORA-IND,X				ORA-Z,Page	ASL-Z,Page		PHP	ORA-IMM	ASL-A			ORA-ABS	ASL-ABS	
1	BPL	ORA-IND,Y				ORA-Z,Page,X	ASL-Z,Page,X		CLC	ORA-ABS,Y				ORA-ABS,X	ASL-ABS,X	
2	JSR	AND-IND,X			BIT-Z,Page	AND-Z,Page	ROL-Z,Page		PLP	AND-IMM	ROL-A		BIT-ABS	AND-ABS	ROL-ABS	
3	BMI	AND-IND,Y				AND-Z,Page,X	ROL-Z,Page,X		SEC	AND-ABS,Y				AND-ABS,X	ROL-ABS,X	
4	RTI	EOR-IND,X				EOR-Z,Page	LSR-Z,Page		PHA	EOR-IMM	LSR-A		JMP-ABS	EOR-ABS	LSR-ABS	
5	BVC	EOR-IND,Y				EOR-Z,Page,X	LSR-Z,Page,X		CLI	EOR-ABS,Y				EOR-ABS,X	LSR-ABS,X	
6	RTS	ADC-IND,X				ADC-Z,Page	ROR-Z,Page		PLA	ADC-IMM	ROR-A		JMP-IND	ADC-ABS	ROR-ABS	
7	BVS	ADC-IND,Y				ADC-Z,Page,X			SEI	ADC-ABS,Y				ADC-ABS,X		
8		STA-IND,X			STY-Z,Page	STA-Z,Page	STX-Z,Page		DEY		TXA		STY-ABS	STA-ABS	STX-ABS	
9	BCC	STA-IND,Y			STY-Z,Page,X	STA-Z,Page,X	STX-Z,Page,Y		TYA	STA-ABS,Y	TXS			STA-ABS,X		
A	LDY-IMM	LDA-IND,X	LDX-IMM		LDY-Z,Page	LDA-Z,Page	LDX-Z,Page		TAY	LDA-IMM	TAX		LDY-ABS	LDA-ABS	LDX-ABS	
B	BCS	LDA-IND,Y			LDY-Z,Page,X	LDA-Z,Page,X	LDX-Z,Page,Y		CLV	LDA-ABS,Y	TSX		LDY-ABS,X	LDA-ABS,X	LDX-ABS,Y	
C	CPY-IMM	CMP-IND,X			CPY-Z,Page	CMP-Z,Page	DEC-Z,Page		INY	CMP-IMM	DEX		CPY-ABS	CMP-ABS	DEC-ABS	
D	BNE	CMP-IND,Y				CMP-Z,Page,X	DEC-Z,Page,X		CLD	CMP-ABS,Y				CMP-ABS,X	DEC-ABS,X	
E	CPX-IMM	SBC-IND,X			CPX-Z,Page	SBC-Z,Page	INC-Z,Page		INX	SBC-IMM	NOP		CPX-ABS	SBC-ABS	INC-ABS	
F	BEQ	SBC-IND,Y				SBC-Z,Page,X	INC-Z,Page,X		SED	SBC-ABS,Y				SBC-ABS,X	INC-ABS,X	

LSD - Least Significant Digit

MSD - Most Significant Digit

6502 Disassembly Table

Special Symbols

A	Accumulator
X, Y	Index Registers
M	Memory
P	Processor Status Register
S	Stack Pointer
✓	Change
—	No Change
+	Add
∧	Logic AND
-	Subtract
∀	Logic Exclusive Or
↑	Transfer from Stack
↓	Transfer to Stack
→	Transfer to
←	Transfer to
∨	Logical OR
PC	Program Counter
PCH	Program Counter High
PCL	Program Counter Low
OPER	OPERAND
#	IMMEDIATE ADDRESSING MODE
$	Indicates a Hex Value

Complete Instruction List with Opcodes

ADC

Add Memory to Accumulator With Carry

Operation: A + M + C → A, C

N	Z	C	I	D	V
✓	✓	✓	–	–	✓

Addressing Mode	Assembly Language Form		OP CODE
Immediate	ADC	# Oper	69
Zero Page	ADC	Oper	65
Zero Page, X	ADC	Oper, X	75
Absolute	ADC	Oper	6D
Absolute, X	ADC	Oper, X	7D
Absolute, Y	ADC	Oper, Y	79
(Indirect, X)	ADC	(Oper, X)	61
(Indirect), Y	ADC	(Oper), Y	71

AND

"AND" Memory With Accumulator

Logical AND to the Accumulator

Operation: A ∧ M → A

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form		OP CODE
Immediate	AND	# Oper	29
Zero Page	AND	Oper	25
Zero Page, X	AND	Oper, X	35
Absolute	AND	Oper	2D
Absolute, X	AND	Oper, X	3D
Absolute, Y	AND	Oper, Y	39
(Indirect, X)	AND	(Oper, X)	21
(Indirect), Y	AND	(Oper), Y	31

ASL

ASL Shift Left One Bit (Memory or Accumulator)

Operation: C ← |7|6|5|4|3|2|1|Ø| ←Ø

N	Z	C	I	D	V
✓	✓	✓	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Accumulator	ASL A	ØA
Zero Page	ASL Oper	Ø6
Zero Page, X	ASL Oper, X	16
Absolute	ASL Oper	ØE
Absolute, X	ASL Oper, X	1E

BCC

BCC Branch on Carry Clear

Operation: Branch on C = Ø

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Relative	BCC Oper	9Ø

BCS

BCS Branch on Carry Set

Operation: Branch on C = 1

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Relative	BCS Oper	BØ

BEQ

BEQ Branch on Result Zero

Operation: Branch on Z = 1

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Relative	BEQ Oper	FØ

BIT

BIT Test Bits in Memory With Accumulator

Operation: $A \wedge M$, $M_7 \rightarrow N$, $M_6 \rightarrow V$

Bit 6 and 7 are transferred to the status Register.

If the result of $A \wedge M$ is zero then Z = 1, otherwise Ø.

N	Z	C	I	D	V
M_7	✓	–	–	–	M_6

Addressing Mode	Assembly Language Form	OP CODE
Zero Page	BIT Oper	24
Absolute	BIT Oper	2C

BMI

BMI Branch on Result Minus

Operation: Branch on N = 1

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Relative	BMI Oper	3Ø

BNE

BNE Branch on Result Not Zero

Operation: Branch on Z = Ø

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Relative	BNE Oper	DØ

BPL

BPL Branch on Result Plus

Operation: Branch on N = Ø

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Relative	BPL Oper	1Ø

BRK

BRK Force Break

Operation: Forced Interrupt PC + 2 ↓ P ↓

N	Z	C	I	D	V
–	–	–	1	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	BRK	ØØ

A BRK command cannot be masked by setting I.

BVC

BVC Branch on Overflow Clear

Operation: Branch on V = 0

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Relative	BVC Oper	5Ø

BVS

BVS Branch on Overflow Set

Operation: Branch on V = 1

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Relative	BVS Oper	7Ø

CLC

CLC Clear Carry Flag

Operation: Ø → C

N	Z	C	I	D	V
–	–	Ø	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	CLC	18

CLD

CLD Clear Decimal Mode

Operation: Ø→D

N	Z	C	I	D	V
–	–	–	–	Ø	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	CLD	D8

CLI

CLI Clear Interrupt Disable Bit

Operation: Ø→I

N	Z	C	I	D	V
–	–	–	Ø	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	CLI	58

CLV

CLV Clear Overflow Flag

Operation: Ø→V

N	Z	C	I	D	V
–	–	–	–	–	Ø

Addressing Mode	Assembly Language Form	OP CODE
Implied	CLV	B8

CMP

CMP Compare Memory and Accumulator

Operation: A - M

N	Z	C	I	D	V
✓	✓	✓	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Immediate	CMP #Oper	C9
Zero Page	CMP Oper	C5
Zero Page, X	CMP Oper, X	D5
Absolute	CMP Oper	CD
Absolute, X	CMP Oper, X	DD
Absolute, Y	CMP Oper, Y	D9
(Indirect, X)	CMP (Oper, X)	C1
(Indirect), Y	CMP (Oper), Y	D1

CPX

CPX Compare Memory and Index X

Operation: X - M

N	Z	C	I	D	V
✓	✓	✓	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Immediate	CPX #Oper	EØ
Zero Page	CPX Oper	E4
Absolute	CPX Oper	EC

CPY

CPY Compare Memory and Index Y

Operation: Y - M

N	Z	C	I	D	V
✓	✓	✓	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Immediate	CPY #Oper	CØ
Zero Page	CPY Oper	C4
Absolute	CPY Oper	CC

DEC

DEC Decrement Memory By One

Operation: M - 1→M

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Zero Page	DEC Oper	C6
Zero Page, X	DEC Oper, X	D6
Absolute	DEC Oper	CE
Absolute, X	DEC Oper, X	DE

DEX

DEX Decrement Index X By One

Operation: X - 1→X

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	DEX	CA

DEY

DEY Decrement Index Y By One

Operation: Y - 1→Y

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	DEY	88

EOR

EOR "Exclusive-OR" Memory With Accumulator

Operation: A ⊻ M→A

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode Mode	Assembly Language Form	OP CODE
Immediate	EOR #Oper	49
Zero Page	EOR Oper	45
Zero Page, X	EOR Oper, X	55
Absolute	EOR Oper	4D
Absolute, X	EOR Oper, X	5D
Absolute, Y	EOR Oper, Y	59
(Indirect, X)	EOR (Oper, X)	41
(Indirect), Y	EOR (Oper), Y	51

INC

INC Increment Memory By One

Operation: M + 1→M

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Zero Page	INC Oper	E6
Zero Page, X	INC Oper, X	F6
Absolute	INC Oper	EE
Absolute, X	INC Oper, X	FE

INX

INX Increment Index X By One

Operation: X + 1→X

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	INX	E8

INY

INY Increment Index Y By One

Operation: Y + 1→Y

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	INY	C8

JMP

JMP Jump to New Location

Operation: (PC + 1)→PCL
(PC + 2)→PCH

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Absolute	JMP Oper	4C
Indirect	JMP (Oper)	6C

JSR

JSR Jump to New Location Saving Return Address

Operation: PC + 2↓, (PC + 1)→PCL
(PC + 2)→PCH

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Absolute	JSR Oper	2Ø

LDA

LDA Load Accumulator With Memory

Operation: M→A

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Immediate	LDA #Oper	A9
Zero Page	LDA Oper	A5
Zero Page, X	LDA Oper, X	B5
Absolute	LDA Oper	AD
Absolute, X	LDA Oper, X	BD
Absolute, Y	LDA Oper, Y	B9
(Indirect, X)	LDA (Oper, X)	A1
(Indirect), Y	LDA (Oper), Y	B1

LDX

LDX Load Index X With Memory

Operation: M→X

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Immediate	LDX #Oper	A2
Zero Page	LDX Oper	A6
Zero Page, Y	LDX Oper, Y	B6
Absolute	LDX Oper	AE
Absolute, Y	LDX Oper, Y	BE

LDY

LDY Load Index Y With Memory

Operation: M→Y

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Immediate	LDY #Oper	AØ
Zero Page	LDY Oper	A4
Zero Page, X	LDY Oper, X	B4
Absolute	LDY Oper	AC
Absolute, X	LDY Oper, X	BC

LSR

LSR Shift Right One Bit (Memory or Accumulator)

Operation: Ø → [7|6|5|4|3|2|1|Ø] → C

N	Z	C	I	D	V
Ø	✓	✓	–	–	–

Addressing Mode	Assembly Language Form		OP CODE
Accumulator	LSR	A	4A
Zero Page	LSR	Oper	46
Zero Page, X	LSR	Oper, X	56
Absolute	LSR	Oper	4E
Absolute, X	LSR	Oper, X	5E

NOP

NOP No Operation

Operation: No Operation (2 cycles)

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	NOP	EA

ORA

ORA "OR" Memory With Accumulator

Operation: A ∨ M → A

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Immediate	ORA #Oper	Ø9
Zero Page	ORA Oper	Ø5
Zero Page, X	ORA Oper, X	15
Absolute	ORA Oper	ØD
Absolute, X	ORA Oper, X	1D
Absolute, Y	ORA Oper, Y	19
(Indirect, X)	ORA (Oper, X)	Ø1
(Indirect), Y	ORA (Oper), Y	11

PHA

PHA Push Accumulator on Stack

Operation: A↓

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	PHA	48

PHP

PHP Push Processor Status on Stack

Operation: P↓

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	PHP	Ø8

PLA

PLA Pull Accumulator From Stack

Operation: A↑

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	PLA	68

PLP

PLP Pull Processor Status From Stack

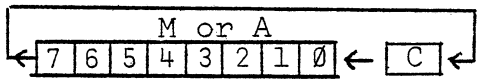
Operation: P↑

N	Z	C	I	D	V
From Stack					

Addressing Mode	Assembly Language Form	OP CODE
Implied	PLP	28

ROL

ROL Rotate One Bit Left (Memory or Accumulator)

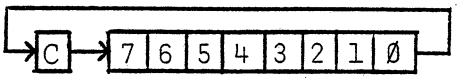
Operation: 

N	Z	C	I	D	V
✓	✓	✓	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Accumulator	ROL A	2A
Zero Page	ROL Oper	26
Zero Page, X	ROL Oper, X	36
Absolute	ROL Oper	2E
Absolute, X	ROL Oper, X	3E

ROR

ROR Rotate One Bit Right (Memory or Accumulator)

Operation: 

N	Z	C	I	D	V
✓	✓	✓	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Accumulator	ROR A	6A
Zero Page	ROR Oper	66
Zero Page, X	ROR Oper, X	76
Absolute	ROR Oper	6E
Absolute, X	ROR Oper, X	7E

RTI

RTI Return From Interrupt

Operation: P↑ PC↑

N	Z	C	I	D	V
From Stack					

Addressing Mode	Assembly Language Form	OP CODE
Implied	RTI	4Ø

RTS

RTS Return From Subroutine

Operation: PC↑, PC + 1 → PC

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	RTS	6Ø

SBC

SBC Subtract Memory From Accumulator With Borrow

Operation: A - M-$\overline{C}$→A

N	Z	C	I	D	V
✓	✓	✓	–	–	✓

Note: $\overline{C}$ = Borrow

Addressing Mode	Assembly Language Form	OP CODE
Immediate	SBC #Oper	E9
Zero Page	SBC Oper	E5
Zero Page, X	SBC Oper, X	F5
Absolute	SBC Oper	ED
Absolute, X	SBC Oper, X	FD
Absolute, Y	SBC Oper, Y	F9
(Indirect, X)	SBC (Oper, X)	E1
(Indirect), Y	SBC (Oper), Y	F1

SEC

SEC Set Carry Flag

Operation: 1→ C

N	Z	C	I	D	V
–	–	1	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	SEC	38

SED

SED Set Decimal Mode

Operation: 1 → D

N	Z	C	I	D	V
–	–	–	–	1	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	SED	F8

SEI

SEI Set Interrupt Disable Status

Operation: 1 → I

N	Z	C	I	D	V
–	–	–	1	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	SEI	78

STA

STA Store Accumulator in Memory

Operation: A→M

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Zero Page	STA Oper	85
Zero Page, X	STA Oper, X	95
Absolute	STA Oper	8D
Absolute, X	STA Oper, X	9D
Absolute, Y	STA Oper, Y	99
(Indirect, X)	STA (Oper, X)	81
(Indirect), Y	STA (Oper), Y	91

STX

STX Store Index X in Memory

Operation: X→M

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Zero Page	STX Oper	86
Zero Page, Y	STX Oper, Y	96
Absolute	STX Oper	8E

STY

STY Store Index Y in Memory

Operation: Y→M

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Zero Page	STY Oper	84
Zero Page, X	STY Oper, X	94
Absolute	STY Oper	8C

TAX

TAX Transfer Accumulator to Index X

Operation: A→X

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	TAX	AA

TAY

TAY Transfer Accumulator to Index Y

Operation: A→Y

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	TAY	A8

TYA

TYA Transfer Index Y to Accumulator

Operation: Y→A

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	TYA	98

TSX

TSX Transfer Stack Pointer to Index X

Operation: S→X

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	TSX	BA

TXA

TXA Transfer Index X to Accumulator

Operation: X→A

N	Z	C	I	D	V
✓	✓	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	TXA	8A

TXS

TXS Transfer Index X to Stack Pointer

Operation: X→S

N	Z	C	I	D	V
–	–	–	–	–	–

Addressing Mode	Assembly Language Form	OP CODE
Implied	TXS	9A

OSI 65V Monitor Mod 2 Listing

ASSEM

```
 10 0000           ; OSI 65U PROM MONITOR  MOD 2
 20 0000           ;
 30 0000           FLAG=$FB
 40 0000           DAT=$FC
 50 0000           PNTL=$FE
 60 0000           PNTH=$FF
 70 0000           ;
 80 FE00                 *=$FE00
 90 FE00 A228      VM    LDX *$28  INITIALIZATION
100 FE02 9A             TXS
110 FE03 D8             CLD
120 FE04 AD06FB         LDA $FB06
130 FE07 A9FF           LDA #$FF
140 FE09 8D05FB         STA $FB05
150 FE0C A2D8           LDX #$D8
160 FE0E A9D0           LDA #$D0
170 FE10 85FF           STA PNTH
180 FE12 A900           LDA #0
190 FE14 85FE           STA PNTL
200 FE16 85FB           STA FLAG
210 FE18 A8             TAY
220 FE19 A920           LDA #'
230 FE1B 91FE     VM1   STA (PNTL),Y
240 FE1D C8             INY
250 FE1E D0FB           BNE VM1
260 FE20 E6FF           INC PNTH
270 FE22 E4FF           CPX PNTH
280 FE24 D0F5           BNE VM1
290 FE26 84FF           STY PNTH
300 FE28 F019           BEQ IN
310 FE2A          ;
320 FE2A 20E9FE   ADDR  JSR INPUT  ADDRESS MODE
330 FE2D C92F           CMP #'/
340 FE2F F01E           BEQ DATA
350 FE31 C947           CMP #'G
360 FE33 F017           BEQ GO
370 FE35 C94C           CMP #'L
380 FE37 F043           BEQ LOAD
390 FE39 2093FE         JSR LEGAL
400 FE3C 30EC           BMI ADDR
410 FE3E A202           LDX #2
420 FE40 20DAFE         JSR ROLL
430 FE43 B1FE     IN    LDA (PNTL),Y
440 FE45 85FC           STA DAT
450 FE47 20ACFE         JSR OUTPUT
460 FE4A D0DE           BNE ADDR
470 FE4C
480 FE4C 6CFE00   GO    JMP (PNTL)
490 FE4F          ;
500 FE4F 20E9FE   DATA  JSR INPUT  DATA MODE
510 FE52 C92E           CMP #'.
```

```
 520 FE54 F0D4           BEQ ADDR
 530 FE56 C90D           CMP #$D
 540 FE58 D00F           BNE DAT4
 550 FE5A E6FE           INC PNTL
 560 FE5C D002           BNE DAT3
 570 FE5E E6FF           INC PNTH
 580 FE60 A000    DAT3   LDY #0
 590 FE62 B1FE           LDA (PNTL),Y
 600 FE64 85FC           STA DAT
 610 FE66 4C77FE         JMP INNER
 620 FE69 2093FE  DAT4   JSR LEGAL
 630 FE6C 30E1           BMI DATA
 640 FE6E A200           LDX #0
 650 FE70 20DAFE         JSR ROLL
 660 FE73 A5FC           LDA DAT
 670 FE75 91FE           STA (PNTL),Y
 680 FE77 20ACFE  INNER  JSR OUTPUT
 690 FE7A D0D3           BNE DATA
 700 FE7C         ;
 710 FE7C 85FB    LOAD   STA FLAG  KICK INPUT DEVICE FLAG
 720 FE7E F0CF           BEQ DATA
 730 FE80         ;
 740 FE80 AD00FC  OTHER  LDA $FB05 UART INPUT SUB.
 750 FE83 4A             LSR A     (FOR AUDIO CASSETTE)
 760 FE84 90FA           BCC OTHER
 770 FE86 AD01FC         LDA $FB03
 780 FE89 EAEAEA         STA $FB07
 790 FE8C 297F           AND #$7F
 800 FE8E 60             RTS
 810 FE8F         ;
 820 FE8F 00             .BYTE 0,0,0,0  EXCESS ROOM
 820 FE90 00
 820 FE91 00
 820 FE92 00
 830 FE93         ;
 840 FE93 C930    LEGAL  CMP #'0  IGNORE NON HEX CHAR.
 850 FE95 3012           BMI FAIL
 860 FE97 C93A           CMP #':
 870 FE99 300B           BMI OK
 880 FE9B C941           CMP #'A
 890 FE9D 300A           BMI FAIL
 900 FE9F C947           CMP #'G
 910 FEA1 1006           BPL FAIL
 920 FEA3 38             SEC
 930 FEA4 E907           SBC #7
 940 FEA6 290F    OK     AND #$F
 950 FEA8 60             RTS
 960 FEA9 A980    FAIL   LDA #$80
 970 FEAB 60             RTS
 980 FEAC         ;
 990 FEAC A203    OUTPUT LDX #3  OUTPUT LLLL  DD
1000 FEAE A000          LDY #0  ONTO SCREEN
```

```
1010 FEB0 B5FC     OUI   LDA DAT, X
1020 FEB2 4A             LSR A
1030 FEB3 4A             LSR A
1040 FEB4 4A             LSR A
1050 FEB5 4A             LSR A
1060 FEB6 20CAFE         JSR DIGIT
1070 FEB9 B5FC           LDA DAT,X
1080 FEBB 20CAFE         JSR DIGIT
1090 FEBE CA             DEX
1100 FEBF 10EF           BPL OUI
1110 FEC1 A920           LDA #'
1120 FEC3 8DCAD0         STA $D0CA
1130 FEC6 8DCBD0         STA $D0CB
1140 FEC9 60             RTS
1150 FECA          ;
1160 FECA 290F     DIGIT AND #$F  OUTPUT 1 DIGIT TO SCREEN
1170 FECC 0930           ORA #$30
1180 FECE C93A           CMP #$3A
1190 FED0 3003           BMI HA1
1200 FED2 18             CLC
1210 FED3 6907           ADC #7
1220 FED5 99C6D0   HA1   STA $D0C6,Y
1230 FED8 C8             INY
1240 FED9 60             RTS
1250 FEDA          ;
1260 FEDA A004     ROLL  LDY #4  MOVE LSD IN AC TO
1270 FEDC 0A             ASL A  LSD IN 2 BYTE NUM.
1280 FEDD 0A             ASL A
1290 FEDE 0A             ASL A
1300 FEDF 0A             ASL A
1310 FEE0 2A       R01   ROL A
1320 FEE1 36FC           ROL DAT,X
1330 FEE3 36FD           ROL DAT+1,X
1340 FEE5 88             DEY
1350 FEE6 D0F8           BNE R01
1360 FEE8 60             RTS
1370 FEE9 A5FB           LDA FLAG  CASSETTE IN?
1380 FEEB D091           BNE $FE7E YES-GO DO ACIA INPUT
1390 FEED 4C00FD         JMP $FD00 NO-GO POLL KB
1400 FEF0 A9FF     KBTEST LDA #$FF KB TEST SUBR.
1410 FEF2 8D00DF         STA $DF00
1420 FEF5 AD00DF         LDA $DF00
1430 FEF8 60             RTS
1440 FEF9 EA             NOP
1450 FEFA 3001           .WORD $130 NMI VECTOR
1460 FEFC 00FE           .WORD $FE00 RESET VECTOR
1470 FEFE C001           .WORD $1C0  IRQ VECTOR
1480 FF00                .END
```

Biblography

1.* How to Program Microcomputers
by William Barden
Howard W. Sams & Company, Inc.
Indianapolis, IN 46268

2. 6502 Software Gourmet Guide and Cookbook
by Robert Findley
SCELBI Publications
20 Hurlbut Street
Elmwood, CT 06110

3. The First Book of KIM

4. Programming a Microcomputer: 6502
by Caxton C. Foster
Addison Wesley Publishing Company, Inc.
Reading, MA 01867

5. 6502 Assembly Language Programming
by Lance Leventhal
Osborne/McGraw-Hill

6. MCS6500 Microcomputer Family Programming Manual
MOS Technology, Inc.
950 Rittenhouse Road
Norristown, PA 19401

7. MICRO: The 6502 Journal
P. O. Box 6502
Chelmsford, MA 01824

8. SY6500/MCS6500 Microcomputer Family Hardware Manual
Synertek
3050 Coronado Drive
Santa Clara, CA 95051

9. Programming the 6502 (Second Edition)
by Rodney Zaks
Sybex
2344 Sixth Street
Berkeley, CA 94710

*Available from OSI

10. 6502 Applications Book
by Rodney Zaks
Sybex
2344 Sixth Street
Berkeley, CA 94710

11. 6502 Games
by Rodney Zaks
Sybex
2344 Sixth Street
Berkeley, CA 94710

Index

absolute address mode - 34, 37, 44, 45
absolute, indexed X (absolute,X) - 33, 34, 37
accumulator - 25
addition - 65
address - 5
address mode - 7, 27
algorithm - 2
AND - 75
arithmetic shift - 71
ASCII code - 14, 88
assembler - 30
assembly - 30

BCD - 16, 82
binary code - 13
bit - 13
byte - 5, 14

C (flag) - 42
carry bit - 37
coding - 11, 24
compare instruction - 44
complier - 25
conditional branch - 44

D (flag) - 82
data mode - 7
debugging - 30
decimal mode - 82
disassembly table - 62

effective address - 27
"execute-in-a-box" - 39

flag - 31, 42
flag (clear) - 42
flag (set) - 42
flowchart - 10

"good listener" - 26

hardware - 5
hexadecimal (hex) - 7

immediate operand - 33, 37
indexed indirect mode - 86
indirect indexed - 81
input data - 2
instruction - 6
instruction register (IR) - 36
instruction set - 24
interpreter - 25

label - 34
LIFO - 57
linking - 85
loader - 85
logical shift - 71

machine language - 3
mask - 75
memory - 5
mnemonic - 27, 90, 91
Monitor - 3

N (flag) - 42, 53
NOP - 38

object code - 3
object language - 3
octal - 19
opcode - 27, 31
operand - 27
OR - 75

P (flag) - 59
pages (memory) - 40
port - 63
processor - 6, 24
program - 2
program counter (PC) - 6, 36, 37
pull, 57, 59
push - 57, 59

radix - 21
RAM - 6
registers - 6
relative address mode - 44
relocatable - 71, 85
ROM - 6

S (stack pointer) - 57
software - 5
source code - 3
source language - 3
stack - 57
subroutine - 61, 79
subtraction - 65

two's complement - 48, 50, 71, 82

V (flag) - 42

X-register - 25

Y-register - 25

Z (flag) - 42, 53, 75
zero page - 40
zero page address mode - 40
zero page indexed - 40